

Informe de Investigación:

El proceso de compilación de Haskell

Florencia Filevich

Universidad ORT - Teoría de la Computación

Introducción

El Glasgow Haskell Compiler (GHC) funciona como un sistema de reescritura que transforma la rica sintaxis de Haskell en un lenguaje intermedio minimalista llamado Core. El objetivo de este informe es documentar la investigación sobre por qué Core no posee una especificación única, realizar pruebas experimentales sobre su salida y analizar su equivalencia con el modelo funcional λ .

El pipeline de compilación y el lenguaje Core

Haskell se reduce a Core a través de un proceso de Desugaring. Core es una implementación del System FC (System F con coerciones).

Anatomía de Core:

Mientras Haskell tiene cientos de reglas, Core se reduce a una sintaxis minimalista:

1. **Variables y Literales:** Identificadores y valores constantes.
2. **Abstracción (lambda):** Definición de funciones.
3. **Aplicación:** Ejecución de cómputo.
4. **Let:** Permite el *sharing* (compartir cómputos para no repetir evaluación).
5. **Case:** El mecanismo de inspección de datos y motor de evaluación estricta.

Experimentación práctica:

Para validar el comportamiento de Core, se realizaron las siguientes pruebas utilizando el compilador GHC directamente desde la terminal.

Prueba 1:

Creemos un archivo llamado Prueba.hs:

Haskell

```
module Prueba where
cuadrado :: Int -> Int
cuadrado n = n * n
```

Para ver el código Core generado por el compilador, ejecutamos el siguiente comando en la terminal:

Bash

```
ghc -O -ddump-simpl -dsuppress-all -dno-typeable-binds Prueba.hs
```

-ddump-simpl muestra el Core optimizado, y los comandos suppress limpian la salida de metadatos irrelevantes para humanos.

La salida del compilador (simplificada) muestra:

```
Haskell
cuadrado = \ (n :: Int) ->
  case n of { l# n_raw ->
    case *# n_raw n_raw of { res_raw ->
      l# res_raw
    }
  }
}
```

Observación: Aquí vemos la esencia de Core. El tipo Int de Haskell es en realidad un "envoltorio" (un objeto en el montículo). El case extrae el l# (el entero crudo o unboxed que la CPU puede entender). La operación *# es una primitiva aritmética directa. Esto demuestra que Core está mucho más cerca del hardware que Haskell.

Prueba 2:

Haskell:

```
longitud [] = 0;

longitud (x:xs) = 1 + longitud xs
```

Core (Deducción):

```
longitud = \ (l :: [a]) ->

  case l of {

    [] -> l# 0# ;

    (x:xs) -> let rec_call = longitud xs

              in case rec_call of { l# n# -> l# (1# +# n#) }

  }
```

Observación: El pattern matching desaparece y se convierte en un único case que inspecciona la estructura de la lista.

Prueba 3:

Haskell:

sumar x y = x + y (donde + pertenece a la clase Num)

Core (Deducción):

```
sumar = \ (dNum :: Num a) (x :: a) (y :: a) ->  
    (+) dNum x y
```

Observación: El polimorfismo se resuelve pasando un argumento extra (dNum), que es un registro con las funciones específicas para ese tipo.

Prueba 4:

Este ejemplo es fundamental porque muestra cómo Core maneja la recursión en estructuras de datos, algo muy similar a lo que vimos en el modelo chi.

Haskell:

```
zeros :: [Int]  
  
zeros = 0 : zeros
```

Core (Deducción):

```
zeros = letrec z = Cons (l# 0#) z in z
```

Observación: Aquí el letrec crea un nodo en el montículo (heap) donde el segundo argumento del constructor Cons apunta a sí mismo. Esto demuestra que en Core, la recursión no solo es para funciones, sino también para datos, permitiendo la existencia de listas infinitas en memoria mediante ciclos de punteros.

Prueba 5:

La composición de funciones es una de las herramientas más usadas en Haskell, pero en Core desaparece para volverse explícita.

Haskell:

```
f :: Int -> Int  
  
f = (+1) . (*2)
```

Core (Deducción):

```
f = \ (x :: Int) ->  
    case x of { l# x_raw ->  
        let mul = *# x_raw 2#
```

```
in l# (+# mul 1#)

}
```

Observación: GHC aplica una optimización llamada inlining. En lugar de llamar a la función de composición (.) y crear funciones intermedias, Core "colapsa" todo en una sola función que recibe el argumento, lo multiplica y luego le suma uno, trabajando directamente con los valores primitivos (#).

Prueba 6:

Haskell:

```
f x = let y = x + 1 in y * y
```

Core:

```
\x -> let y = x + 1# in y *# y
```

Equivalente en chi (usando case para simular let):

```
\x -> case (x + 1) of { y -> y * y }
```

Conclusión: Este mapeo prueba que las directivas adicionales de Core son conveniencias para el compilador, pero no alteran la semántica fundamental del lenguaje.

Deducción del BNF de Core

A partir de la experimentación y los ejemplos anteriores, podemos deducir que la sintaxis abstracta de Core responde a la siguiente gramática:

BNF

<exp> ::= <var>	-- Variables
<literal>	-- Valores básicos
\ <var> :: <type> -> <exp>	-- Abstracción
<exp> <exp>	-- Aplicación
let <var> = <exp> in <exp>	-- Ligadura simple
letrec <var> = <exp> in <exp>	-- Recursión
case <exp> of { <ramas> }	-- Evaluación y decisión

| <exp> `cast` <coercion> -- Coerciones

<alt> ::= <con> <vars> -> <exp> -- Rama de case

Comparación con especificaciones de Internet:

Al contrastar la gramática deducida de nuestras pruebas con la documentación oficial de la Wiki de GHC y el paper System F with Type Equality Coercions, se observan las siguientes conclusiones:

- El BNF obtenido en la experimentación es plenamente consistente con la estructura de los 6 constructores básicos del System FC. Esto valida que el Core generado por GHC es, efectivamente, una implementación técnica y pragmática de dicho sistema formal.
- Las especificaciones formales en Internet (especialmente en los papers de Peyton Jones) ponen mucho énfasis en las coerciones mediante el constructor `cast` e `by co`, que sirve como evidencia de igualdad de tipos. En nuestras pruebas de terminal, estas evidencias suelen estar ocultas o simplificadas debido al uso del flag `-dsuppress-all`, que limpia la salida para que el código sea legible para humanos.
- Mientras que el BNF teórico del System FC suele ser genérico y abstracto, el Core real de GHC está "contaminado" positivamente con tipos `unboxed` (identificados por el sufijo `#`, como `Int#` o `*#`). Estos tipos representan datos crudos de la CPU y son fundamentales para asegurar el rendimiento en aplicaciones reales, marcando una diferencia clave entre el modelo académico y la implementación industrial.

El problema de la especificación

Una de las preguntas centrales de esta investigación es: ¿Por qué no hay una especificación oficial y única de Core?

Core no es un estándar internacional como el lenguaje C. Es una herramienta **interna** de GHC. Como Haskell sigue evolucionando (agregando tipos lineales, tipos dependientes, etc.), Core debe cambiar en cada versión para dar soporte a estas innovaciones. Un estándar fijo mataría la capacidad de GHC de seguir siendo un compilador de vanguardia.

Además, existe una brecha entre el System FC que los investigadores publican en papers (que es elegante y puramente matemático) y el Core real que maneja el compilador. El Core real tiene "basura técnica" necesaria para que los programas vuelen: anotaciones sobre si una variable se usa mucho o poco (strictness analysis), punteros específicos y manejos de memoria que no aparecen en los modelos teóricos.

Diferencias entre versiones: Las especificaciones existentes difieren en cómo manejan los tipos `unboxed` (los que llevan `#`) y las reglas de reescritura (Rewrite Rules), lo que hace que un manual de Core de la versión 8.x sea impreciso para la 9.x.

Core vs. chi:

Al analizar el BNF de Core obtenido en la experimentación y compararlo con el modelo funcional chi, se llega a las siguientes conclusiones:

- **Funciones:** La abstracción lambda de chi es idéntica en comportamiento a la de Core.
- **Recursión:** Mientras que en chi usamos el constructor rec, en Core se utiliza el letrec (una variante de let que permite que el nombre definido se use dentro de su propia definición).
- **Constructores y Case:** Ambos sistemas confían en el case como el motor principal de cómputo. Sin case, ninguno de los dos lenguajes podría "tomar decisiones" o desarmar datos.

Traducción Expresiva: De Core a chi:

Para demostrar que son equivalentes a nivel expresivo, podemos mapear cada directiva de Core a una construcción en chi

Directiva Core	Construcción en χ
$\lambda x \rightarrow e$	$\lambda x.e$
$e1\ e2$	$e1e2$
$\text{let } x = e1 \text{ in } e2$	$(\lambda x.e2)e1$
$\text{letrec } f = e1 \text{ in } e2$	$e2[f := \text{rec } f.e1]$
$\text{case } e \text{ of } \{ K\ xs \rightarrow e' \}$	$\text{case } e \text{ of } \{ K \rightarrow xs. e' \}$
$l\#1\#$ (Literales)	Succ (Zero)

Análisis de la directiva let y el sharing:

Es importante notar que, aunque Core utiliza la directiva let para mejorar la performance mediante el sharing (evitando re-evaluar expresiones), a nivel puramente expresivo el let es redundante. En el modelo chi (o incluso en el cálculo lambda puro), un $\text{let } x = e1 \text{ in } e2$ puede simularse mediante un case o una aplicación: $\text{case } e1 \text{ of } \{ x \rightarrow e2 \}$. Esto demuestra que la presencia de let en Core es una decisión de optimización técnica y no una necesidad de poder de cómputo.

Tipado vs. Generalidad:

Mientras que Core es un sistema estrictamente tipado (basado en System FC), el modelo chi es un sistema sin tipos (untyped). Desde la teoría de la computación, no tener tipos es más general que tenerlos, ya que un lenguaje sin tipos permite expresar comportamientos que un sistema de tipos estricto podría rechazar. Al quitarle las anotaciones de tipo a Core,

lo que obtenemos es esencialmente un programa en chi, reafirmando que Core es una restricción segura y eficiente de la libertad que ofrece chi.

Conclusión de la equivalencia:

En resumen, cualquier algoritmo que puedas escribir en chi tiene una representación directa en Core. La gran diferencia es que chi es "puro" (diseñado para estudiar qué es computable), mientras que Core es "enriquecido" con un sistema de tipos polimórfico complejo y optimizaciones de hardware. Son dos caras de la misma moneda: uno para la academia y otro para la industria.

Conclusión

La investigación permite concluir que el lenguaje Core es mucho más que un paso intermedio en la compilación, es el nexo donde convergen la teoría funcional pura y la ejecución eficiente de bajo nivel. A través de la experimentación práctica, se demostró cómo GHC destila abstracciones complejas en una estructura minimalista que, a pesar de carecer de una especificación única por su naturaleza evolutiva, mantiene una coherencia absoluta con los pilares de la computación. La validación de su consistencia con modelos formales como chi confirma que el proceso de compilación de Haskell no es una transformación arbitraria, sino una reducción sistemática hacia los fundamentos de la computabilidad. Si bien Core introduce elementos pragmáticos como los tipos unboxed y las coerciones para optimizar el rendimiento en hardware real, su corazón lógico permanece idéntico a los sistemas de reescritura de términos estudiados en la academia. En última instancia, Core demuestra que es posible construir un lenguaje industrial de alto nivel sobre una base matemática rigurosa, garantizando que la elegancia del código fuente no sacrifique la potencia del ejecutable final.

Bibliografía

- Peyton Jones, S. (2007). *System F with Type Equality Coercions*.
- GHC Wiki: *The Core Type System*.
- Stanford University CS242: *Contracting Haskell to Core*.